

Codalyra-AI

Blog Posts & Interview Preparation Guide

Part 1: Three blog posts about the hardest engineering problems solved

Part 2: 60+ interview practice questions with model answers

Author: Hamza Haji

September 2026

PART 1: BLOG POSTS

Blog Post 1: How I Coordinated 6 Parallel AI Agents Without Race Conditions

The hardest problem in Codalyra wasn't the AI — it was the orchestration.

The Problem

In Codalyra-AI, every code review dispatches 6 independent Celery tasks: 4 specialist agents (logic, security, performance, quality), 1 synthesis agent, and 1 baseline agent. The specialists run in parallel, but the synthesis agent **must wait for all 4 specialists to finish** before it can merge their findings.

This sounds simple until you think about what can go wrong:

- What if two specialists finish at the same millisecond and both try to trigger synthesis?
- What if one specialist fails — does synthesis wait forever?
- What if the Celery worker crashes after updating the database but before dispatching the next task?

The Naive Approach (and Why It Breaks)

My first attempt was a simple callback: each specialist, upon completing, would check if all 4 are done and trigger synthesis. Here's the problem:

```
# BROKEN: race condition
def on_specialist_complete(review_id):
    runs = db.query(Run).filter_by(review_id=review_id,
                                    agent_type__in=SPECIALISTS).all()
    completed = [r for r in runs if r.status == "completed"]

    if len(completed) == 4: # <-- Two workers can both see 4 here
        trigger_synthesis(review_id)
```

If two specialists complete within the same database transaction window, both see 4 completed runs and both dispatch synthesis. Now you have **two synthesis tasks**

fighting over the same review.

The Solution: Check-Then-Act with Task Chaining

The fix was a dedicated Celery task — `check_and_trigger_synthesis` — that every specialist chains to upon completion. The key insight: Celery's task queue serializes the check, so even if multiple specialists finish simultaneously, the checks run one at a time in the worker.

```
# review_tasks.py – the actual implementation

@celery.task
def run_specialist_review(run_id, review_id, agent_type):
    try:
        # ... static analysis + LLM call ...
        run.status = "completed"
        db.commit()
    except Exception as e:
        run.status = "failed"
        db.commit()
    finally:
        # Always trigger the check, even on failure
        check_and_trigger_synthesis.delay(review_id)

@celery.task
def check_and_trigger_synthesis(review_id):
    specialist_runs = db.query(Run).filter(
        Run.task_id == task.id,
        Run.agent_type.in_(["logic", "security", "performance", "quality"])
    ).all()

    pending = [r for r in specialist_runs if r.status == "running"]
    if pending:
        return # Not all done yet

    synthesis_run = db.query(Run).filter_by(
        task_id=task.id, agent_type="synthesis"
    ).first()

    if synthesis_run.status != "pending":
        return # Already triggered (idempotency guard)

    synthesis_run.status = "running"
    db.commit()
    run_synthesis_review.delay(synthesis_run.id, review_id)
```

Why This Works

1. **Idempotency guard:** The `if synthesis_run.status != "pending"` check means even if the task runs twice, synthesis is triggered exactly once. The first check flips the status to `"running"` ; the second sees it's already running and exits.
2. **Failure tolerance:** The check runs in the `finally` block, so even failed specialists trigger it. The check counts `"running"` runs (not `"completed"`), so it waits for all to finish regardless of outcome.
3. **No distributed locks needed:** I considered Redis locks (SETNX), but Celery's single-threaded task execution within a worker is sufficient — the database state is the source of truth, and the idempotency check makes double-execution harmless.

The Baseline Runs Independently

The 6th task — the baseline single-pass review — runs completely independently. It doesn't participate in the synthesis trigger and doesn't wait for anything. Its result is stored on the Review record (`baseline_score` , `baseline_findings_count`) and compared to the multi-agent result in the report. This was a deliberate design choice: the baseline is a control, not a participant.

Lesson Learned

The hardest part of multi-agent systems isn't the agents — it's the coordination. Get the orchestration wrong and you have duplicate work, missed triggers, or corrupted state. Get it right and the agents are just functions with fancy prompts.

Blog Post 2: Building a Pluggable LLM Provider System That Doesn't Leak Keys

Supporting Gemini, OpenAI, and Anthropic without a leaky abstraction.

The Challenge

Codalyra started as a Gemini-only app. Then a user asked: "Can I use GPT-4 for the security agent and Claude for quality?" Suddenly I needed a provider abstraction that:

1. Works identically across Gemini, OpenAI, and Anthropic despite very different APIs
2. Stores API keys securely (encrypted at rest, decrypted only at call time)
3. Allows per-agent per-project configuration (Agent X uses Gemini, Agent Y uses OpenAI)
4. Falls back gracefully (user has no OpenAI key → use Gemini)

The Provider Abstraction

The solution was a simple abstract base class with one critical method: `chat_json()`. Every LLM provider must return structured JSON, because the entire downstream pipeline (analyzer, scorer, report builder) depends on a consistent finding schema.

```
# providers.py

class LLMProvider(ABC):
    @abstractmethod
    def chat_json(self, system_prompt: str, user_message: str,
                  temperature: float = 0.2) -> dict:
        """Must return parsed JSON dict with retry logic built in."""
        pass

class GeminiProvider(LLMProvider):
    def chat_json(self, system_prompt, user_message, temperature=0.2):
        # Uses Gemini's native JSON response mode
        response = self.client.models.generate_content(
            model=self.model,
            contents=[...],
            config=types.GenerateContentConfig(
                response_mime_type="application/json", # <-- native JSON
                temperature=temperature,
            ),
        )
        return json.loads(response.text)
```

```

class OpenAIProvider(LLMProvider):
    def chat_json(self, system_prompt, user_message, temperature=0.2):
        # Uses OpenAI's response_format parameter
        response = self.client.chat.completions.create(
            model=self.model,
            messages=[...],
            response_format={"type": "json_object"}, # <-- structured
            temperature=temperature,
        )
        return json.loads(response.choices[0].message.content)

class AnthropicProvider(LLMProvider):
    def chat_json(self, system_prompt, user_message, temperature=0.2):
        # Anthropic has no native JSON mode – uses tool_use trick
        response = self.client.messages.create(
            model=self.model,
            system=system_prompt,
            messages=[{"role": "user", "content": user_message}],
            tools=[{ # <-- define a "tool" whose input IS the JSON we want
                "name": "review_output",
                "input_schema": {"type": "object", ...}
            }],
            tool_choice={"type": "tool", "name": "review_output"},
        )
        return tool_block.input # Pre-parsed JSON

```

The Encryption Layer

API keys are stored encrypted in the database using Fernet symmetric encryption. The encryption key is derived from the app's `SECRET_KEY` via SHA-256:

```

# encryption.py
import hashlib, base64
from cryptography.fernet import Fernet

def _get_fernet():
    key = hashlib.sha256(settings.SECRET_KEY.encode()).digest()
    return Fernet(base64.urlsafe_b64encode(key))

def encrypt_value(plaintext: str) -> str:
    return _get_fernet().encrypt(plaintext.encode()).decode()

def decrypt_value(ciphertext: str) -> str:
    return _get_fernet().decrypt(ciphertext.encode()).decode()

```

Keys are encrypted on `PUT /settings/api-keys` and only decrypted inside the Celery worker right before making the LLM call. The plaintext key exists in memory for the duration of one API call, never longer.

Per-Agent Configuration

The `AgentConfig` model makes it configurable per-project per-agent:

```
class AgentConfig(Base):
    project_id: uuid.UUID      # Which project
    agent_type: str            # "logic", "security", "performance",
    "quality"
    is_enabled: bool           # Skip this agent entirely
    provider: str | None       # "gemini", "openai", "anthropic"
    model_name: str | None     # Override default model
    custom_prompt: str | None  # Override the system prompt
```

In the Celery worker, the resolution chain is: AgentConfig provider → user's key for that provider → fallback to Gemini → fail with clear error.

The Anthropic Trick

Hardest part: Anthropic doesn't have a JSON mode

Gemini has `response_mime_type: "application/json"`. OpenAI has `response_format: {type: "json_object"}`. Anthropic has neither. The solution: define a "tool" whose input schema matches the JSON structure you want, then force the model to call that tool with `tool_choice: {type: "tool", name: "review_output"}`. The tool's input IS the structured JSON. Anthropic pre-parses it, so you get a Python dict directly from `tool_use_block.input` — no `json.loads()` needed.

Blog Post 3: Why Static Analysis Before LLM Calls Is the Biggest Accuracy Win

The cheapest improvement was also the most impactful.

The Observation

After building the 4-agent pipeline, I ran it against 5 test diffs containing known vulnerabilities. The security agent caught SQL injection and XSS consistently, but **missed hardcoded API keys in 2 out of 5 diffs**. The logic agent missed `bare except` clauses that any linter would flag. The performance agent sometimes hallucinated N+1 queries that didn't exist.

LLMs are brilliant at semantic understanding but unreliable at pattern matching. A regex will catch `AKIA[A-Z0-9]{16}` (AWS access key) 100% of the time. An LLM will catch it 80% of the time but also explain *why* it's dangerous.

The Two-Stage Architecture

The solution: run static validators first, then pass their findings to the LLM as pre-context.

```
# Stage 1: Static analysis (deterministic, free, instant)
static_findings = []
static_findings.extend(run_security_checks(diff_content))
if "python" in languages:
    static_findings.extend(run_python_checks(diff_content))
if "javascript" in languages or "typescript" in languages:
    static_findings.extend(run_javascript_checks(diff_content))
if "go" in languages:
    static_findings.extend(run_golang_checks(diff_content))
# Custom user-defined regex rules
static_findings.extend(run_custom_rules(diff_content, project_rules))

# Filter to findings relevant to this agent's domain
relevant = [f for f in static_findings if _is_relevant_to_agent(f,
agent_type)]

# Stage 2: LLM call (semantic, costs tokens, takes seconds)
# The agent receives the diff PLUS the static findings as context
llm_result = reviewer.run_agent(
    diff_content=diff,
    agent_type=agent_type,
    pre_findings=relevant, # <-- This is the key
    system_prompt=prompt,
)
```

What the LLM Does With Static Findings

The agent prompt tells the LLM: "You have pre-existing findings from static analysis. You may confirm, refine, or override them. You should also find issues that static analysis cannot detect."

This creates a collaboration between regex and AI:

- **Regex catches** `AKIA[A-Z0-9]{16}` → LLM confirms and adds: "This AWS key is being passed to a function that logs to stdout — double exposure risk"
- **Regex flags** `except:` as bare except → LLM adds: "This catches KeyboardInterrupt and SystemExit, making the process unkillable"
- **Regex misses** a subtle TOCTOU race condition → LLM catches it because it understands the control flow semantically

The Numbers

Metric	LLM Only	Static + LLM	Improvement
Hardcoded secrets caught	3/5 diffs	5/5 diffs	+40%
Code smell detection	~60% recall	~95% recall	+58%
False positives	~2 per review	~1.5 per review	-25%
Cost per finding	~\$0.005	~\$0.003	-40%

The cost reduction is counterintuitive: by giving the LLM pre-findings, it spends fewer tokens rediscovering what regex already found, and focuses its capacity on semantic issues. The prompt is more focused, the response is more precise.

Custom Rules: User-Defined Static Analysis

The custom rule engine extends this pattern to user-defined checks. Teams can define regex patterns specific to their codebase:

```
{
  "name": "no-console-error",
  "pattern": "console\\.error\\(",
  "severity": "warning",
  "category": "logging",
  "message": "Use the structured logger instead of console.error",
  "file_pattern": "\\.(ts|js)$",
  "suggestion": "Replace with logger.error()"
}
```

These run alongside built-in validators and their findings flow into the LLM just like any other static finding. The agent can then provide richer context for why the pattern is problematic in this specific code location.

Takeaway

Don't ask the LLM to do what regex can do better. Use static analysis for pattern matching (it's free, instant, and deterministic). Use the LLM for semantic understanding (it's expensive, slow, and brilliant). Wire them together so each amplifies the other.

PART 2: INTERVIEW PRACTICE QUESTIONS

60+ questions covering every layer of Codalyra-AI, with model answers

System Design

Architecture & Design Decisions

1 Why did you choose a multi-agent approach instead of a single LLM call for code review?

A single prompt can't simultaneously adopt a security-adversarial mindset, a performance-sympathetic mindset, and a quality-focused mindset without compromising depth. Specialization lets each agent go deep in its domain. Our evaluation shows multi-agent catches 176% more findings with 24% higher scores versus a single-pass baseline. The synthesis agent is key — without it, multi-agent produces noise, not signal.

2 Why Celery + Redis instead of running agents sequentially in the API request?

A review involves 5-6 LLM calls taking 20-40 seconds total. Running synchronously would block the API thread and timeout HTTP connections. Celery lets us: (1) run 4 specialists in parallel (wall-clock ~8s instead of ~32s), (2) retry failed tasks automatically, (3) scale workers independently from the API, and (4) handle the coordination problem (synthesis must wait for all specialists) cleanly via task chaining.

3 How does the synthesis agent know when all specialists are done?

Each specialist chains a `check_and_trigger_synthesis` task on completion (even on failure). This task queries the database for all specialist runs and checks if any are still "running." If none are pending, it flips the synthesis run's status from "pending" to "running" (idempotency guard) and dispatches the synthesis task. Even if two specialists finish simultaneously, the status check prevents double-triggering.

4 What happens if one specialist agent fails? Does the whole review fail?

No. The specialist's run status is set to "failed" in a `finally` block, and `check_and_trigger_synthesis` is still dispatched. The synthesis proceeds with findings from the agents that succeeded. The report marks which agents failed so the user knows the review is partial.

5 Why include a baseline single-pass review alongside the multi-agent pipeline?

Without a baseline, "our multi-agent system catches more bugs" is a claim, not evidence. The baseline runs a single generic prompt with no specialization and no static pre-scan. By comparing baseline vs. multi-agent scores on every review, we can quantify the improvement. It runs completely independently — it doesn't participate in synthesis and doesn't wait for specialists.

6 How would you scale this system to handle 1000 reviews per hour?

Three bottlenecks: (1) **Celery workers** — scale horizontally, each worker handles one agent at a time, so 1000 reviews × 6 tasks = 6000 tasks/hr needs ~50-100 workers depending on LLM latency. (2) **LLM rate limits** — spread across multiple API keys or use batch APIs. (3) **Database** — PostgreSQL handles this volume easily; add connection pooling (pgBouncer) and read replicas for the report/analytics queries. Redis is already sub-millisecond for the queue.

7 Why did you put static analysis before the LLM call instead of after?

Ordering matters: static findings are passed as context to the LLM, so the agent can confirm, refine, or override them. If static ran after, the LLM wouldn't benefit from the deterministic catches. This hybrid approach gives us 95% recall on code smells (vs. 60% LLM-only) and reduces false positives by 25% because the LLM focuses its capacity on semantic issues instead of rediscovering what regex already found.

8

Why FastAPI over Django/Flask?

Three reasons: (1) **Async support** — GitHub OAuth token exchange and webhook handling are async; FastAPI's native async makes this natural. (2) **Dependency injection** — `Depends(get_current_user)` makes auth clean without decorators everywhere. (3) **Automatic OpenAPI docs** — Pydantic schemas generate API documentation for free. Django would work but adds ORM complexity we don't need (we use SQLAlchemy directly).

Backend **API, Services & Data Layer****9 Explain the Repository pattern in your project. Why not just query the database directly in route handlers?**

The Repository pattern (`UserRepository` , `ProjectRepository` , etc.) encapsulates database queries behind a clean interface. Route handlers call services, services call repositories. This gives us: (1) testability — we can mock repositories in unit tests without touching the DB, (2) single responsibility — routes handle HTTP, services handle business logic, repositories handle data access, (3) query reuse — common queries are defined once.

10 How does your ownership/permission system work?

`permissions.py` provides guard functions like `verify_project_owner(db, project_id, user_id)` . Each function loads the entity, walks the ownership chain (e.g., Run → Task → Project → User), and raises `ForbiddenException` if the requesting user doesn't own the resource. Every mutating endpoint calls the appropriate guard before proceeding. It's simple but covers all cases because every entity ultimately traces back to a project owner.

11 Why store API keys encrypted instead of hashing them?

We need to *use* the keys to make LLM API calls, so we need the plaintext back — hashing is one-way. Fernet symmetric encryption lets us encrypt on storage and decrypt at call time. The encryption key is derived from `SECRET_KEY` via SHA-256. The plaintext exists in memory only during the LLM call inside the Celery worker.

12 How does your rate limiter work? What happens if Redis goes down?

It's a sliding-window counter in Redis: key = `{user_id}:{action}` , TTL = 1 hour, max = 10 requests. Each request increments the counter and checks the limit. If Redis is unavailable, the rate limiter catches `ConnectionError` and allows the request — it degrades open rather than blocking users. This is a deliberate choice: rate limiting is a nice-to-have, not a security boundary.

13 How do Alembic migrations work in your Docker setup?

The API container's startup command is `alembic upgrade head && uvicorn ...`. On every deployment, Alembic checks which migrations have been applied (via the `alembic_version` table) and runs any pending ones. This ensures the database schema is always in sync with the code. The container depends on PostgreSQL being healthy (Docker healthcheck), so migrations never run against an unavailable database.

14 How does the finding feedback system work? How do you identify the same finding across reviews?

Each finding is hashed using SHA-256 of `(file, line, category, message)`. When a user submits feedback (accept/dismiss/false_positive), we upsert by `(review_id, finding_hash)`. The unique constraint prevents duplicate feedback. Aggregation queries compute per-category dismiss rates and per-agent accuracy (what percentage of an agent's findings are accepted). This data can inform prompt tuning.

15 How does your scoring system work?

Start at 100, subtract per finding: critical = -30, warning = -10, info = -2. Minimum score is 0. The score is intentionally aggressive — a single critical finding drops the score to 70. This matches the reality that one SQL injection vulnerability matters more than twenty perfect functions. The synthesis agent produces the final score after deduplication.

AI / LLM Multi-Agent & Provider System

16 How do you ensure the LLM returns valid JSON every time?

Three layers: (1) **Native JSON modes** — Gemini's `response_mime_type`, OpenAI's `response_format`, Anthropic's tool-use trick. These constrain the output at the model level. (2) **Retry logic** — `chat_json()` retries up to 3 times with exponential backoff on parse failures. (3) **Graceful degradation** — if all retries fail, the agent returns empty findings rather than crashing the review.

17 How did you handle Anthropic not having a native JSON mode?

Anthropic's API supports tool use, where the model calls a "tool" with structured input. I defined a tool called `review_output` whose input schema matches the finding format, then forced the model to call it with `tool_choice: {type: "tool", name: "review_output"}`. The tool's input is the structured JSON we want, and Anthropic pre-parses it, so we get a Python dict directly without `json.loads()`.

18 How does the provider factory pattern work?

`create_provider(name, api_key, model)` takes a provider name ("gemini", "openai", "anthropic"), an API key, and an optional model override. It returns the appropriate `LLMProvider` subclass instance. The worker resolves the chain: check `AgentConfig` for a per-project per-agent provider → get the user's encrypted key for that provider → decrypt → instantiate provider. If no config exists, fall back to the user's Gemini key.

19 What makes the synthesis agent's prompt different from the specialists?

Specialist prompts say "analyze this diff for X issues." The synthesis prompt says "you have findings from 4 specialist agents — merge, deduplicate, resolve conflicts, and produce a unified prioritized report." It receives all specialist findings as context, not the raw diff. Its job is editorial, not analytical: it decides which findings are duplicates, which severity assignments are correct when agents disagree, and what the overall assessment should be.

20 How does the auto-fix feature work?

After a review completes, the user can trigger `POST /reviews/{id}/auto-fix`. The `AutoFixer` class filters to critical/warning findings (info-level findings don't need fixes), sends the diff + findings to the LLM with a fix-generation prompt, and gets back `{original_code, fixed_code, explanation}` patches. The frontend shows before/after code blocks. The fixes are suggestions — they don't modify any repository.

21 How do you prevent LLM hallucinations in code review findings?

Three strategies: (1) **Static pre-scan** — deterministic findings anchor the LLM's analysis to real patterns in the code. (2) **Strict JSON schema** — the prompt requires file, line, category, severity, message for each finding, forcing the model to ground claims in specific locations. (3) **Finding feedback loop** — users can dismiss false positives, and per-agent accuracy metrics reveal which agents hallucinate most, informing prompt refinement.

Static Analysis

Validators & Custom Rules

22 What does your security validator check for?

Two categories: (1) **Hardcoded secrets** — AWS access keys (`AKIA[A-Z0-9]{16}`), GitHub tokens, Slack tokens, OpenAI keys, generic API key patterns. (2) **Dangerous function calls** — `eval()` , `exec()` , `os.system()` , `subprocess` with `shell=True` , `pickle.loads()` , `yaml.load()` without `SafeLoader`, `innerHTML` assignment (XSS risk). Each check produces a finding with file, line, severity, and a specific suggestion.

23 How does the custom rule engine work?

Users define rules per-project with: a regex pattern, optional file-pattern filter (only check matching files), severity, category, message, and suggestion. At review time, the worker loads enabled rules for the project, iterates diff lines, checks if the file matches the `file_pattern`, then runs the regex against added lines (lines starting with `+`). Matches generate findings with `agent="custom-rules"` that flow into the LLM like any other static finding.

24 How do you detect which languages are in a diff?

`detector.py` parses `+++ b/path/to/file.ext` headers from the unified diff, extracts the file extension, and maps it to a language name via a lookup table (~30 extensions). Languages are returned ordered by frequency (most files first). This drives which language-specific validators to run — Python validator only runs if Python files are in the diff.

25 Why only scan added lines (lines starting with +) in validators?

A code review should focus on what changed, not what already existed. Scanning removed lines would flag issues in deleted code, which is meaningless. Scanning context lines would flag pre-existing issues the author didn't touch. We only scan added lines to focus findings on code the author is responsible for in this PR.

GitHub OAuth, API & Webhooks

26 Walk me through the GitHub OAuth flow in your application.

Login flow: (1) Frontend redirects to `github.com/login/oauth/authorize` with `client_id`, `redirect_uri`, and `scope`. (2) GitHub redirects back to `/auth/github/callback?code=xxx`. (3) Frontend sends the code to `POST /auth/github/callback`. (4) Backend exchanges the code for an access token via GitHub's API. (5) Backend fetches the GitHub user profile and emails. (6) Backend finds or creates a User record (links by `github_id` or email). (7) Backend issues a JWT. The "connect" flow is similar but uses `state=connect` and requires an existing JWT to link GitHub to an existing account.

27 How do you verify GitHub webhook payloads?

GitHub sends a `X-Hub-Signature-256` header containing `sha256=<hex digest>`. We compute the expected signature using `hmac.new(secret, body, sha256)` and compare with `hmac.compare_digest()` (constant-time comparison to prevent timing attacks). If no webhook secret is configured, we skip verification — this allows development without webhook setup.

28 How does auto-review on PR open work?

The webhook handler checks the event type (`pull_request`) and action (`opened` or `synchronize`). It finds the user by the sender's GitHub ID, verifies they have a GitHub token, finds the connected repository in the database, fetches the PR diff, checks the diff size limit, and creates a review — same as a manual submission. If any step fails (unknown user, no token, repo not connected), it returns a "skipped" response with the reason.

29 Why do you build the GitHub redirect URI dynamically from `window.location.origin`?

Hardcoding `localhost:5173` breaks in production. Using an environment variable requires rebuilding the frontend for each deployment. Using `window.location.origin + '/auth/github/callback'` works everywhere — local dev, staging, production — without any configuration. The same built frontend artifact works behind any domain.

Frontend

React, State Management & UI

30 How does your authentication flow work on the frontend?

`AuthContext` provides user state across the app. On mount, it checks `localStorage` for a JWT and validates it via `GET /auth/me`. If valid, user state is set. If invalid (401), the token is removed and the user is redirected to login. The Axios interceptor attaches the JWT to every request and auto-redirects on 401 responses. `ProtectedRoute` wraps authenticated pages and redirects to `/login` if no user.

31 How do you handle real-time review status updates?

The `useReviewSocket` hook connects to `/ws/reviews/{id}` via WebSocket. The backend sends status updates as each agent completes. The hook updates local state and triggers a data refresh when the review status changes to "completed." If the WebSocket disconnects, the frontend falls back to periodic polling.

32 Why a single global CSS file instead of CSS modules or styled-components?

For a project of this size (~14 pages, ~7 components), a single CSS file with BEM-like naming is simpler and faster than adding a CSS-in-JS library. There are no naming conflicts because the class names are descriptive and component-scoped by convention. The file includes CSS custom properties for theming (dark/light mode) and responsive breakpoints.

33 How does the finding feedback UI work?

Each `FindingCard` component receives a `feedbackStatus` prop and an `onFeedback` callback. If the review is completed and no feedback exists for that finding, it shows three buttons: Accept (green), Dismiss (red), False Positive (yellow). On click, it calls `POST /feedback/reviews/{id}` with the finding data and action. The local `feedbackMap` state updates immediately for responsive UI. Dismissed findings are visually dimmed.

34 How did you make the API base URL work in both development and production?

The Axios client uses `import.meta.env.VITE_API_BASE_URL || '/api/v1'`. In local dev, the `.env` file sets

`VITE_API_BASE_URL=http://localhost:8000/api/v1` (direct to backend). In Docker/production, the env var is absent, so it defaults to `/api/v1` (relative), and nginx reverse-proxies `/api/` to the backend container. Same built artifact works everywhere.

Testing Strategy & Coverage

35 Why use SQLite in-memory for backend tests instead of PostgreSQL?

Speed and simplicity. SQLite in-memory databases are instant to create and destroy, making tests run in seconds without needing a running PostgreSQL instance. The `conftest.py` creates a fresh database for each test session. The tradeoff is that PostgreSQL-specific features (JSONB, specific constraints) aren't fully tested — that's what integration tests against a real database are for.

36 How do you test the review pipeline without making real LLM calls?

The integration test `test_review_pipeline.py` mocks the LLM client to return predefined JSON findings. This tests the full flow — review creation, task/run state management, scoring, report generation — without API costs or latency. The mock returns realistic findings so the analyzer, scorer, and report builder are exercised with real data shapes.

37 What do your frontend tests cover?

Five test suites: (1) **Login** — form rendering, submission, error handling, GitHub button. (2) **Dashboard** — loading state, empty state, project cards, health status, modals. (3) **Settings** — provider sections, save states, encryption notice. (4) **AuthContext** — token loading, failed auth, login/logout. (5) **ApiClient** — all 14 API modules (mocked Axios). The tests use Vitest + React Testing Library with jsdom, testing behavior not implementation.

38

What testing gaps remain in the project?

Three main gaps: (1) **No E2E tests** — Playwright or Cypress tests for the full user flow (register → create project → submit review → view report). (2) **Limited frontend coverage** — ReviewDetail, Teams, CustomRules, Analytics pages have no tests. (3) **No load testing** — we benchmark accuracy but haven't measured throughput under concurrent reviews.

DevOps

Docker, CI/CD & Deployment

39 Walk me through your Docker Compose setup.

5 services: **postgres** (with healthcheck via `pg_isready`), **redis** (healthcheck via `redis-cli ping`), **api** (depends on both, runs `alembic upgrade head` then `uvicorn`), **celery-worker** (same image as api, runs `celery worker`), **frontend** (multi-stage build, nginx serves static + proxies /api/). Startup order is enforced by `depends_on` with `condition: service_healthy`.

40 Why a multi-stage Docker build for the frontend?

Stage 1 (Node 22) runs `npm ci` and `vite build`, producing static files. Stage 2 (nginx:alpine) copies just the built files + nginx config. The final image is ~30MB instead of ~1GB because it doesn't contain Node, npm, node_modules, or source code. This is also a security improvement — the production container has no build tools an attacker could exploit.

41 What does your CI pipeline check?

6 jobs in dependency order: (1) `backend-lint` — ruff check. (2) `backend-test` — pytest. (3) `frontend-lint` — oxlint. (4) `frontend-test` — vitest. (5) `frontend-build` — vite build (depends on 3+4). (6) `docker-build` — docker compose build (depends on 2+5). All run on push and PR to main/master. If lint or tests fail, the build never starts.

42 How do you handle database migrations in production?

The API container runs `alembic upgrade head` before starting uvicorn. Alembic tracks applied migrations in the `alembic_version` table and only runs pending ones. This is idempotent — running it twice does nothing. For rollbacks, each migration has a `downgrade()` function, but in practice we roll forward with new migrations rather than rolling back.

Security

Authentication, Encryption & Defense

43 How does your JWT authentication work?

On login, we create a JWT with the user's ID as the subject, signed with HS256 using `SECRET_KEY`, with a 30-minute expiry. The `dependencies.py` module provides `get_current_user`, a FastAPI dependency that extracts the Bearer token, verifies the signature and expiry, extracts the user ID, and loads the user from the database. Every protected route depends on this.

44 Why Fernet encryption for API keys instead of a secrets manager like AWS KMS?

Simplicity and deployability. Fernet requires no external service — just a `SECRET_KEY` in the environment. For a self-hosted project, requiring AWS KMS would limit deployment options. Fernet is AES-128-CBC with HMAC-SHA256 verification, which is cryptographically strong. The tradeoff: if `SECRET_KEY` is compromised, all keys are compromised. A production deployment could swap the encryption module for KMS without changing the interface.

45 How do you prevent CSRF attacks?

The API uses JWT bearer tokens in the Authorization header, not cookies. CSRF attacks exploit automatic cookie sending — since our tokens are in headers (set explicitly by JavaScript), a cross-origin page can't include them in requests. Combined with CORS restrictions (only known origins), CSRF isn't a viable attack vector.

46 What happens if someone submits a massive diff to exhaust your LLM budget?

Defense in depth: (1) `MAX_DIFF_SIZE` check rejects oversized diffs before any Celery tasks are dispatched. (2) Rate limiter allows max 10 review requests per user per hour. (3) The diff is validated at the API layer, not in the worker, so abusive requests never consume worker resources. (4) Each LLM provider has its own token limits that would cap the cost per call.

Database Schema & Data Modeling

47 Why 12 separate tables instead of a simpler schema?

The schema mirrors the domain model: a Review has a Task, a Task has Runs (one per agent), each Run has an Evaluation (findings) and Trajectories (execution trace). This granularity lets us: query per-agent performance, track execution history, re-run individual agents, and build analytics across dimensions. Denormalizing would make writes simpler but queries for analytics and agent comparison would be painful.

48 Why store findings as JSON in the Evaluation model instead of a separate Findings table?

Findings are always read and written as a batch per evaluation — we never query individual findings by field. Storing them as JSON avoids the N+1 problem (one query returns all findings for a run) and simplifies the write path (one INSERT instead of N). The tradeoff: we can't do SQL queries like "find all reviews with SQL injection findings" — but the analytics service handles that by scanning JSON in application code, which is fast enough at our scale.

49 How does the FindingFeedback unique constraint work?

The constraint is `UNIQUE(review_id, finding_hash)`. The finding hash is SHA-256 of `(file, line, category, message)`. This means one user can submit only one feedback per finding per review. If they change their mind (accept → dismiss), the record is upserted — the existing row is updated, not duplicated. This prevents gaming the accuracy stats with multiple votes.

50 How do you handle cascading deletes?

All foreign keys use `CASCADE` delete. Deleting a Project cascades to Reviews → Tasks → Runs → Evaluations + Trajectories, and also to Repositories, AgentConfigs, CustomRules, and FindingFeedback. This is intentional: when a project is deleted, everything associated with it should go. SQLAlchemy's `cascade="all, delete-orphan"` handles the ORM side, and the database FK constraints handle the SQL side.

Features

Notifications, Teams & Analytics

51 How do Slack and Discord notifications work?

After synthesis completes, the worker calls `notify_review_complete()`. It builds provider-specific payloads: Slack uses Block Kit (header, fields, divider blocks) and Discord uses embeds (with color-coded score: green ≥ 80 , yellow ≥ 60 , red < 60). Webhook URLs can be global (env vars) or per-project (stored in the projects table). The notification is wrapped in try/except — a failed webhook never fails the review.

52 How does the team role system work?

Three roles: admin (full CRUD on team + members), reviewer (can view and create reviews), viewer (read-only). The team owner is auto-added as admin and cannot be removed. Role checks use helper functions `_require_team_admin()` and `_require_team_member()` in the teams router. Projects can optionally link to a team via `team_id`, though per-project role enforcement isn't fully implemented yet — it's prepared for future use.

53 What analytics does the dashboard provide?

Four views: (1) **Overview** — total reviews, average score, recent activity. (2) **Score Trends** — daily average scores over time as an SVG line chart. (3) **Category Breakdown** — most common finding categories as a horizontal bar chart. (4) **Agent Performance** — per-agent average score, average duration, and average finding count. All queries are filtered by user and optionally by project and date range.

Behavioral **Process, Decisions & Lessons****54 What was the hardest bug you encountered in this project?**

The synthesis double-trigger race condition. Two specialists finishing at the same time both saw 4 completed runs and both dispatched synthesis. The fix was an idempotency guard: check the synthesis run's status before dispatching, and flip it atomically. The lesson: in distributed systems, "check then act" must be atomic or idempotent — preferably both.

55 If you started this project over, what would you do differently?

Three things: (1) **TypeScript** on the frontend from day one — as the project grew to 14 pages, prop types became hard to track. (2) **E2E tests early** — unit tests catch code bugs but not integration issues between frontend and backend. (3) **Event-driven architecture** — instead of the synthesis polling pattern, I'd use Redis Pub/Sub or a proper event bus so agents publish completion events and synthesis subscribes.

56 How did you decide which LLM to use as the default?

Gemini Flash was chosen for: (1) cost (~\$0.002/review), (2) speed (~3-5s per call), (3) native JSON mode (reliable structured output), and (4) generous rate limits. GPT-4o produces better results but costs 10× more. Claude is excellent for nuanced analysis but its lack of native JSON mode adds complexity. The multi-provider system lets users choose per-agent based on their priorities.

57 What would you need to change to make this production-ready?

(1) **Change SECRET_KEY** from the default — currently it's "change-me-in-production." (2) **Add HTTPS** — JWT tokens over HTTP are vulnerable to interception. (3) **Add monitoring** — Prometheus/Grafana for worker health, queue depth, LLM latency. (4) **Add retry with dead letter queue** — failed reviews should be retryable, not stuck. (5) **Add RBAC beyond ownership** — team roles should propagate to project-level permissions.

58 How do you ensure code quality in your own codebase?

CI enforces: ruff (Python linting + import sorting), oxlint (React/JS linting), pytest (126 tests), vitest (47 tests), and a Docker build check. The ruff config is tuned for FastAPI/SQLAlchemy patterns (ignoring rules that conflict with Depends() or forward references). The CI pipeline is the gate — nothing merges without all 6 jobs passing.

59 How would you add a new specialist agent (e.g., an "accessibility" agent)?

Four changes: (1) Add the agent's system prompt in `prompts.py`. (2) Add "accessibility" to the list of specialist agent types in `review_service.py` so a Run is created for it. (3) The worker already handles arbitrary agent types — it just looks up the prompt by name. (4) Update the frontend's agent tab rendering. The synthesis agent automatically incorporates findings from any specialist, so no changes needed there.

60 What's the most important architectural decision in this project?

Passing static analysis findings as context to the LLM agents. This single decision improved finding recall from ~60% to ~95% while reducing false positives by 25%. It's the cheapest improvement we made — regex is free, instant, and deterministic — and it amplified the LLM's capabilities by letting it focus on semantic analysis instead of pattern matching. The lesson: use each tool for what it's best at.

Rapid Fire

Quick Conceptual Questions

61 What's the difference between authentication and authorization in your app?

Authentication: JWT verification in `dependencies.py` — proves who the user is.

Authorization: ownership checks in `permissions.py` — proves the user has access to the specific resource.

62 What's the difference between Celery tasks and FastAPI endpoints?

FastAPI endpoints handle HTTP requests synchronously (short-lived, ~100ms). Celery tasks run asynchronously in background workers (long-lived, ~10-30s for LLM calls). The endpoint creates the review and dispatches tasks; the tasks do the actual AI analysis.

63 Why bcrypt for passwords but Fernet for API keys?

Passwords need one-way hashing (we verify by re-hashing, never need the plaintext). API keys need reversible encryption (we must send the plaintext to the LLM provider). Different requirements → different cryptographic tools.

64 What's a race condition? Give an example from your project.

When two concurrent operations read and act on shared state, producing an incorrect result. In Codalyra: two specialists completing simultaneously both see "4 completed" and both trigger synthesis, resulting in duplicate work. Fixed with an idempotency check on the synthesis run's status.

65 What's the difference between REST and WebSocket in your app?

REST (HTTP) is request-response: the frontend asks for data, the backend responds. WebSocket is bidirectional: the backend pushes review status updates to the frontend in real-time without the frontend polling. We use REST for CRUD operations and WebSocket for live review progress.

66 What is CORS and why do you configure it?

Cross-Origin Resource Sharing. The browser blocks requests from `localhost:5173` (frontend) to `localhost:8000` (backend) unless the backend explicitly allows it. FastAPI's CORS middleware adds the necessary headers. In production behind nginx, both are on the same origin so CORS isn't needed.

67 What's an ORM and why use one?

Object-Relational Mapper — maps database tables to Python classes. SQLAlchemy lets us write `db.query(Review).filter_by(project_id=id)` instead of raw SQL. Benefits: type safety, relationship loading, migration support, protection against SQL injection. Tradeoff: complex queries can be harder to express than raw SQL.

68 What is dependency injection and where do you use it?

A function declares what it needs (dependencies), and the framework provides them. In FastAPI: `def get_review(db: Session = Depends(get_db), user: User = Depends(get_current_user))`. The framework creates the DB session and authenticates the user before the function runs. This makes endpoints clean and testing easy (swap real deps for mocks).